Terraform indepth learning

1, Terraform ante enti?
Definition: Terraform is an **Infrastructure as Code (IaC)** tool used to **define, provision, and manage infrastructure using configuration files/code instead of manually creating and managing infrastructure**.
Terraform ante **Infrastructure ni code tho manage cheyyadaniki use chese tool**.
Simple ga cheppalante:
"AWS/Azure/GCP lo servers, databases, networks manually create cheyyakunda, code rasI create/manage cheyyadam" = Terraform.
Oka real example
Suppose nuvvu AWS lo oka application deploy cheyyali.
Manual ga chesthe:

- EC2 server create cheyyali
- VPC create cheyyali
- Subnet create cheyyali
- Security Group create cheyyali
- Load Balancer create cheyyali
- Database create cheyyali
- Permissions configure cheyyali

Ivi anni AWS Console lo click-click chesi cheyyachu.
Kani problem enti ante **time padutundi, mistakes avvachu, same setup ni malli create cheyyadam difficult.**
Terraform tho:

```
resource "aws_instance" "app_server" {
  ami        = "ami-xxxxxxxx"
  instance_type = "t3.micro"
}
```

Then:

```
terraform init
terraform plan
terraform apply
```

Terraform AWS ki cheppi:
"Naku ilaanti EC2 server kavali."
AWS lo create chestundi.

Terraform enduku vadali?
Main reason **automation + consistency + repeatability**.
Example:
Today production environment create chesav.
Repu same architecture tho staging environment kavali.
Manual ga malli anni create cheyyali.
Terraform lo same code ni variables tho reuse cheyyachu.

```
Terraform Code
   ↓
 AWS API
   ↓
Infrastructure
```

Inka important benefits:

- **Automation** — manual clicking taggutundi
- **Repeatability** — same infrastructure malli create cheyyachu
- **Version control** — Terraform code ni Git lo maintain cheyyachu
- **Review** — infrastructure changes ni PR lo review cheyyachu
- **Consistency** — dev/staging/prod environments consistent ga maintain cheyyachu
- **Multi-cloud** — AWS, Azure, GCP etc. manage cheyyachu
- **Change tracking** — infrastructure lo em changes jarugutunnayo telustundi

---

Eppudu Terraform vadali?
Terraform especially useful when infrastructure **complex or repeatable** ga unnappudu.
For example:

```
AWS
 ├── VPC
 ├── Subnets
 ├── EC2
 ├── Load Balancer
 ├── RDS
 ├── S3
 ├── IAM
 └── Security Groups
```

Ivi manually create cheyyadam badulu Terraform code lo define cheyyachu.
Small personal project
Oka EC2 instance create chesi test cheyyadaniki Terraform **necessary kaadu**.
AWS Console lo create chesina saripovachu.
Company / production environment
Multiple servers, databases, networking, security, staging/prod environments unte Terraform **chaala useful**.

Terraform ela work avtundi?
Terraform lo important concept:
Desired State
Nuvvu Terraform ki:
"Na infrastructure ila undali."
ani code lo cheptav.
Terraform current infrastructure ni check chesi, desired state ki teesukeltundi.

```
Terraform Code
      ↓
Desired State
      ↓
terraform plan
      ↓
What will change?
      ↓
terraform apply
      ↓
Cloud Provider
      ↓
Actual Infrastructure
```

Terraform lo 4 commands first nerchuko
1. terraform init
Project initialize chestundi.
terraform init
Usually first command.

---

2. terraform plan
Terraform em changes cheyyabotundo chupistundi.
terraform plan
Example:
+ create aws_instance.app_server
Meaning:
Oka EC2 instance create cheyyabotundi.

---

3. terraform apply
Actual ga infrastructure create/change chestundi.
terraform apply

---

4. terraform destroy
Terraform create chesina infrastructure remove cheyyachu.
terraform destroy
Important: Production lo destroy command careless ga run cheyyakudadhu.

Terraform file ela untundi?
Usually .tf files use chestaru.
Example:

```
terraform-project/
│
├── main.tf
├── variables.tf
├── outputs.tf
└── terraform.tfvars
```

Simple main.tf:

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "app" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t3.micro"
}
```

Here:
provider "aws"

means:
Terraform AWS tho work cheyyali.
And:
resource "aws_instance" "app"
means:
AWS lo EC2 instance ane resource kavali.

Terraform vs Docker vs Kubernetes
Idi chala important distinction.
Terraform
Infrastructure create/manage cheyyadaniki
VPC
EC2
RDS
S3
Load Balancer
IAM
Docker
Application ni containerize cheyyadaniki
My Java App
↓
Docker Image
↓
Container
Kubernetes
Containers ni run/manage cheyyadaniki
Kubernetes
↓
Pods
↓
Containers
So typical DevOps architecture:
Terraform
↓
AWS Infrastructure
↓
Kubernetes
↓
Docker Containers
↓
Your Application

Terraform nerchukovadaniki correct order
Nuvvu beginner aithe direct advanced Terraform ki vellakunda ila velladam best:
1. Linux basics
↓
2. Git/GitHub
↓
3. AWS basics
↓
4. Terraform basics
↓
5. Terraform variables
↓
6. outputs
↓
7. state
↓
8. modules
↓
9. remote state
↓
10. Terraform + CI/CD
Terraform start cheyyadaniki **AWS basics konchem teliste chaala help avtundi**.
One-line definition gurthupettuko:
Terraform is an Infrastructure as Code (IaC) tool used to define, provision, and manage infrastructure using configuration files instead of manually creating resources.
Nuvvu **DevOps beginner** aithe, next nenu Terraform ni **zero nunchi practical ga — AWS EC2 create cheyyadam → variables → state → modules → real company project structure** varaku Telugu lo step-by-step teach cheyyagalanu.

2. What is Infrastructure as Code (IaC)?

Definition:

Infrastructure as Code (IaC) is the practice of defining and managing infrastructure using code or configuration files instead of manually creating and configuring infrastructure.

Simple ga cheppalante:

“Infrastructure ni manual ga create cheyyakunda, code/configuration tho create and manage cheyyadam” = IaC.

Terraform is one of the popular tools used for Infrastructure as Code.

Oka real example:

Suppose oka company ki application run cheyyadaniki:

- VPC
- Subnets
- EC2
- Load Balancer
- Database
- Security Groups

kavali.

Manual ga:

AWS Console
↓
Create VPC
↓
Create Subnets
↓
Create EC2
↓
Create Load Balancer
↓
Create Database
↓
Configure Security

IaC use chesthe:

Infrastructure Code
↓
Tool
↓
Cloud Provider
↓
Infrastructure

Terraform tho aa infrastructure ni configuration files lo define cheyyachu.

IaC enduku use chestam?

Main reason:

Automation + Consistency + Repeatability

Suppose:

Development
Testing
Production

Three environments ki similar infrastructure kavali.

Manual ga three times create chesthe configuration differences or mistakes ravachu.

IaC tho same code/configuration ni reuse chesi environments create cheyyachu.

IaC valla benefits:

- **Automation** — Manual infrastructure creation taggutundi
- **Consistency** — Same configuration repeat cheyyachu
- **Repeatability** — Infrastructure ni malli create cheyyachu
- **Version Control** — Infrastructure code ni Git lo store cheyyachu
- **Review** — Infrastructure changes ni code review cheyyachu
- **Documentation** — Code itself infrastructure definition laga untundi
- **Disaster Recovery** — Infrastructure ni code nunchi recreate cheyyachu

Simple example:

Suppose production lo:

3 EC2 Servers
1 RDS
1 Load Balancer

Undi.

Same architecture staging lo kavali.

Manual ga:

Again create everything manually

IaC tho:

Terraform Code

↓
Variables
↓
Staging Configuration
↓
Staging Infrastructure
Same code ni different values tho reuse cheyyachu.
Terraform and IaC relationship:
Infrastructure as Code
↓
Terraform
↓
Cloud Infrastructure
IaC anedi **concept/practice**.
Terraform anedi **IaC implement cheyyadaniki use chese tool**.
One-line definition gurthupettuko:
Infrastructure as Code = Managing infrastructure using code/configuration instead of manually creating and configuring resources.

---

3. Why do we use Terraform?
Terraform ni mainly **infrastructure creation and management automate cheyyadaniki** use chestam.
Manual infrastructure management lo time ekkuva padutundi and configuration mistakes jaragachu.
Terraform infrastructure ni code lo define chesi, same configuration ni repeatedly use cheyyadaniki help chestundi.
Simple ga cheppalante:
"Infrastructure ni fast ga, consistently, repeatedly and automatically create/manage cheyyadaniki Terraform use chestam."
Oka real example:
Suppose oka e-commerce application ki:
VPC
↓
3 EC2 Servers
↓
Load Balancer
↓
RDS Database
kavali.
Manual ga create chesthe:
AWS Console
↓
Create VPC
↓
Create Subnets
↓
Create EC2
↓
Configure Load Balancer
↓
Create RDS
Chala steps untayi.
Terraform tho:
Terraform Code
↓
terraform plan
↓
terraform apply
↓
Infrastructure
Terraform required resources ni create/manage chestundi.
Terraform use cheyyadam valla main benefits:
1. Automation
Manual ga resources create cheyyalsina requirement taggutundi.
Code
↓
Terraform
↓
Infrastructure
2. Consistency
Same configuration use chesthe environments consistent ga maintain cheyyachu.
Same Code
↓
Dev
↓

Testing
↓
Production
3. Repeatability
Oka infrastructure setup ni malli recreate cheyyachu.
Example:
Production infrastructure:
VPC
EC2
RDS
ALB
Same architecture staging lo kuda create cheyyachu.
4. Version Control
Terraform code ni Git lo store cheyyachu.
Terraform Code
↓
Git
↓
Version History
Evaru em change chesaro track cheyyachu.
5. Code Review
Infrastructure changes ni application code laga review cheyyachu.
Developer
↓
Terraform Change
↓
Pull Request
↓
Code Review
↓
Apply
6. Easy Change Management
Suppose:
EC2 Servers = 3
nunchi:
EC2 Servers = 5
ki change cheyyali.
Terraform configuration update chesi plan/apply cheyyachu.
Terraform required changes ni identify chestundi.
Eppudu Terraform useful?
Especially:

- Company infrastructure
- Production environments
- Multiple environments
- Large infrastructure
- Repeatable infrastructure
- CI/CD pipelines
- Team-based infrastructure management

Terraform use cheyyakunda simple ga cheyyagalige situation:
Suppose testing kosam just oka EC2 instance create cheyyali.
AWS Console lo manually create cheyyadam easy ayye situation lo Terraform compulsory kaadu.
Kani infrastructure grow ayye koddi Terraform value ekkuva avtundi.
Simple flow:
Terraform Code
↓
Automation
↓
Infrastructure Creation
↓
Infrastructure Changes
↓
Infrastructure Management
One-line answer:
Terraform = Automates, standardizes, and manages infrastructure using code.

---

4. What is Terraform Architecture?
Terraform Architecture ante **Terraform configuration nunchi actual infrastructure varaku Terraform ela work chestundo** explain chese structure.
Terraform architecture lo main components:

- Terraform Configuration
- Terraform Core
- Terraform State
- Terraform Providers
- Cloud Provider / API
- Actual Infrastructure

Simple architecture:

Terraform Configuration
↓
Terraform Core
↓
Provider
↓
Cloud API
↓
Infrastructure

1. Terraform Configuration

Manam .tf files lo infrastructure ni define chestam.
Example:
main.tf
variables.tf
outputs.tf
providers.tf
Configuration lo manam:

- Resources
- Variables
- Providers
- Outputs
- Modules

define cheyyachu.

2. Terraform Core

Terraform Core Terraform configuration ni process chestundi.
It understands:

- Desired configuration
- State
- Dependencies
- Required changes

Terraform Core determine chestundi:
“Current infrastructure nunchi desired infrastructure ki velladaniki em changes cheyyali?”

3. Terraform Provider

Provider Terraform ki external platform/API tho communicate cheyyadaniki help chestundi.
Examples:
Terraform → AWS Provider → AWS
Terraform → Azure Provider → Azure
Terraform → Google Provider → GCP
Terraform → Kubernetes Provider → Kubernetes

4. Cloud API

Provider target platform APIs tho communicate chestundi.
Example:
Terraform
↓
AWS Provider
↓
AWS API
↓
EC2 / VPC / RDS / ALB

5. Terraform State

Terraform state Terraform managed infrastructure gurinchi information maintain chestundi.
State valla Terraform ki:
“Nenu already em resources manage chestunnanu?”
ane information telustundi.
Oka real example:
Suppose AWS lo e-commerce application undi.
Required:
VPC
↓
EC2

↓
Load Balancer
↓
RDS
Terraform code:
Terraform Configuration
↓
Terraform Core
↓
AWS Provider
↓
AWS API
↓
AWS Infrastructure
Terraform configuration says:
EC2 = 3
Terraform current state/infrastructure ni evaluate chesi required action determine chestundi.
Terraform Architecture lo State role:
Configuration
↓
Terraform Core
↕
State
↓
Provider
↓
Cloud API
↓
Infrastructure
Main point:
Terraform Core configuration and state ni process chesi, Providers dwara external platforms/APIs tho communicate chesi infrastructure ni manage chestundi.
One-line interview answer:
Terraform Architecture consists of Terraform configuration, Terraform Core, state, providers, and cloud APIs, which together allow Terraform to manage infrastructure.

---

5. What is Terraform Workflow?
Terraform Workflow ante **Terraform configuration ni write chesi, infrastructure changes ni validate, preview, apply and manage chese sequence**.
Common Terraform workflow:
Write
↓
Initialize
↓
Validate
↓
Plan
↓
Apply
↓
Manage
Step 1: Write Terraform Configuration
First infrastructure requirement ni .tf files lo define chestam.
Example:
main.tf
variables.tf
outputs.tf
providers.tf
Suppose:
EC2
RDS
Load Balancer
create cheyyali.
Vatini Terraform configuration lo define chestam.
Step 2: terraform init
Project ni initialize cheyyadaniki:
terraform init
Usually Terraform project lo first command.
It prepares the working directory and installs required providers/modules and initializes the backend when configured.
Flow:
Terraform Code

↓
terraform init
↓
Providers / Modules / Backend
↓
Ready

Step 3: terraform validate

Configuration correct ga unda check cheyyadaniki:
terraform validate
Configuration syntax and internal consistency ni validate cheyyadaniki help chestundi.
Terraform Code
↓
terraform validate
↓
Valid / Error

Step 4: terraform plan

Terraform em changes cheyyabotundo preview cheyyadaniki:
terraform plan
Example:
Current:
EC2 = 3
Desired:
EC2 = 5
Plan lo Terraform approximately:
+ Create 2 EC2 instances
ani chupistundi.
Important:
terraform plan actual infrastructure changes cheyyadu.

Step 5: terraform apply

Plan/configuration ni actual infrastructure meeda apply cheyyadaniki:
terraform apply
Then:
Terraform
↓
Provider
↓
Cloud API
↓
Create / Update / Delete
↓
Infrastructure

Step 6: Manage Infrastructure

Infrastructure create ayyaka kuda Terraform use chestam.
Suppose:
3 EC2
nunchi:
5 EC2
ki change cheyyali.
Configuration update chesi:
terraform plan
terraform apply
run cheyyachu.

Step 7: terraform destroy

Environment completely remove cheyyali ante:
terraform destroy
Terraform managed resources ni destroy cheyyadaniki use chestam.
Production lo destroy command very carefully use cheyyali.

Complete Terraform Workflow:

Write Terraform Code
↓
terraform init
↓
terraform validate
↓
terraform plan
↓
Review Changes
↓
terraform apply
↓
Infrastructure

↓
Modify / Manage
↓
terraform plan
↓
terraform apply
If infrastructure no longer required:
Infrastructure
↓
terraform destroy
↓
Resources Deleted
Real-time CI/CD workflow:
Company environment lo usually:
Developer
↓
Git
↓
CI/CD Pipeline
↓
terraform fmt
↓
terraform validate
↓
terraform plan
↓
Approval
↓
terraform apply
↓
Cloud Infrastructure
Main point:
Terraform Workflow = Write → Initialize → Validate → Plan → Review → Apply → Manage.
One-line interview answer:
Terraform workflow is the process of writing configuration, initializing Terraform, validating the code, reviewing the execution plan, applying the changes, and managing the infrastructure.

---

6. What is a Terraform Configuration File?

Definition:
A Terraform Configuration File is a file containing Terraform configuration that defines infrastructure and Terraform settings.
Terraform configuration files normally use the .tf **file extension.**
Simple meaning:
“.tf file lo manaki kavalsina infrastructure ni code form lo define chestam.”
For example:
main.tf
variables.tf
outputs.tf
providers.tf
All these can contain Terraform configuration.

Why do we use Terraform Configuration Files?
Because Terraform ki:
“Naku exactly elaanti infrastructure kavali?”
ani configuration files dwara cheptam.
For example:
EC2
Instance Type = t3.micro
or:
VPC
Subnets
Security Groups
Load Balancer

Where are Terraform Configuration Files used?
Terraform project directory lo .tf files untayi.
Example:
terraform-project/
│

├── main.tf
├── variables.tf
├── outputs.tf
├── providers.tf
└── terraform.tfvars
Terraform working directory lo unna .tf configuration files ni Terraform process chestundi.

Common Terraform Files
main.tf
Usually main resources define cheyyadaniki use chestam.
Example:
EC2
VPC
RDS
variables.tf
Input variables define cheyyadaniki.
Example:
region
instance_type
environment
outputs.tf
Useful information output cheyyadaniki.
Example:
EC2 Public IP
Load Balancer DNS
Resource ID
providers.tf
Provider configuration kosam.
Example:
AWS Provider
terraform.tfvars
Variable values provide cheyyadaniki commonly use chestam.

Real-time Example
Suppose AWS lo EC2 create cheyyali.
main.tf:

```
resource "aws_instance" "app" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t3.micro"
}
```

This configuration says:
“AWS lo t3.micro type EC2 instance kavali.”
Terraform configuration:
main.tf
↓
Terraform
↓
AWS Provider
↓
AWS
↓
EC2

How do we use the configuration?
Configuration create chesina tarvatha normal workflow:
main.tf
↓
terraform init
↓
terraform validate
↓
terraform plan
↓
terraform apply
Why each command?

- terraform init → Project dependencies and providers prepare cheyyadaniki.
- terraform validate → Configuration valid ga unda check cheyyadaniki.
- terraform plan → What changes will happen ani choodadaniki.
- terraform apply → Actual infrastructure create/change cheyyadaniki.

Important Point
Terraform project lo files separate ga unna, Terraform వాటిని one configuration ga treat chestundi.
For example:
main.tf
variables.tf
outputs.tf
providers.tf
Terraform ee .tf files ni read chesi configuration ni understand chestundi.

Simple Flow
Terraform .tf Files
↓
Terraform
↓
Provider
↓
Infrastructure
Interview Main Point:
A Terraform Configuration File is a .tf **file used to define infrastructure and Terraform configuration such as resources, providers, variables, and outputs.**

---

7. What is a Terraform Provider?
Definition:
A Terraform Provider is a plugin that allows Terraform to communicate with an external platform or service through its API and manage its resources.
Simple meaning:
"Terraform ki AWS, Azure, Google Cloud, Kubernetes lanti platforms tho communicate cheyyadaniki Provider bridge laga work chestundi."

Why do we need a Provider?
Terraform itself directly every cloud platform ni understand cheyyadu.
For example, AWS lo EC2 create cheyyali ante Terraform ki AWS API tho communicate cheyyali.
AWS Provider aa communication ki required functionality provide chestundi.
Terraform
↓
AWS Provider
↓
AWS API
↓
EC2

Where are Providers used?
Different platforms ki different providers untayi.
Examples:
Terraform
├— AWS Provider → AWS
├— Azure Provider → Azure
├— Google Provider → Google Cloud
└— Kubernetes Provider → Kubernetes
So Provider depends on the platform we want to manage.

How does a Provider work?
Suppose we want an EC2 instance.
Step 1
Terraform configuration lo AWS provider declare chestam.
```
provider "aws" {
 region = "ap-south-1"
}
```
This tells Terraform:
"AWS platform tho work cheyyali."
Step 2
Resource define chestam.
```
resource "aws_instance" "app" {
 ami        = "ami-xxxxxxxx"
 instance_type = "t3.micro"
}
```
Step 3
Terraform AWS Provider ni use chestundi.
Terraform Configuration
↓
Terraform

↓
AWS Provider
↓
AWS API
↓
EC2 Instance

Real-time Example
Suppose e-commerce application AWS lo run avtundi.
Required resources:
VPC
EC2
RDS
Load Balancer
S3
Terraform AWS Provider ni use chesi AWS resources ni manage cheyyachu.
Terraform
↓
AWS Provider
↓
AWS
├— VPC
├— EC2
├— RDS
├— Load Balancer
└— S3

Important Commands
terraform init
Provider ni download/install chesi Terraform project ki ready chestundi.
Terraform Configuration
↓
terraform init
↓
AWS Provider Downloaded
↓
Ready
Why do we use it?
Terraform configuration lo required provider dependencies ni initialize cheyyadaniki.

---

terraform plan
Provider ni use chesi current infrastructure information tho desired configuration ni compare chesi planned changes ni show chestundi.
terraform plan
Why?
AWS lo em changes jaragabotunnayo ముందే choodadaniki.

---

terraform apply
Provider dwara actual AWS API calls chesi infrastructure changes ni apply chestundi.
terraform apply
Why?
Actual AWS resources create/update/delete cheyyadaniki.

Provider vs Resource
This is an important interview question.
Provider
Platform/service tho communication kosam.
Example:
AWS Provider
Resource
Actual infrastructure object ni represent chestundi.
Example:
aws_instance
Simple ga:
Provider
↓
AWS tho communicate chestundi

Resource
↓
AWS lo EC2 represent chestundi

Simple Flow

Terraform Code

↓

Terraform

↓

Provider

↓

API

↓

Cloud Platform

↓

Infrastructure

Interview Main Point:

A Terraform Provider is a plugin that enables Terraform to communicate with an external platform or service through APIs and manage its resources.